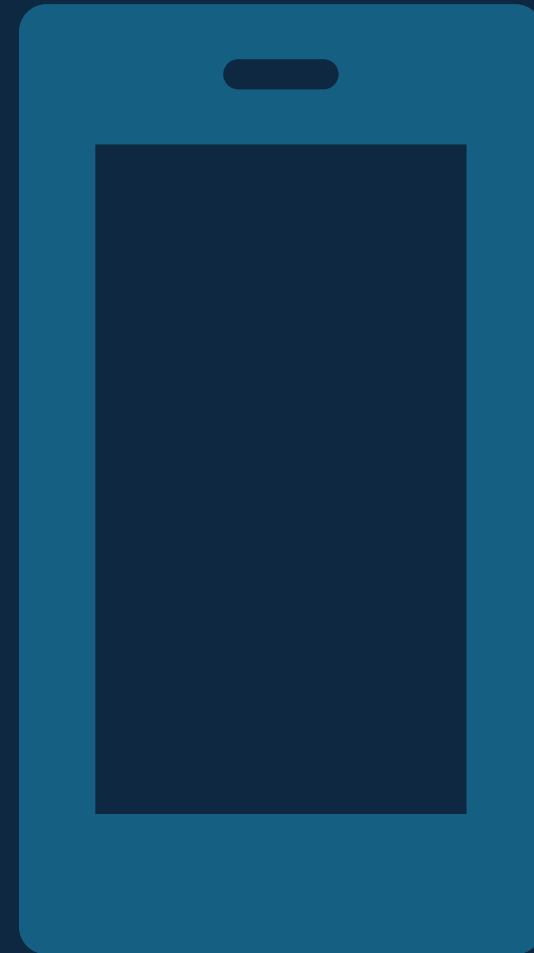


Javascript – BASE PARA O DESENVOLVIMENTO COM REACT

Material
MOBILE_002





DOM

- Nesta aula veremos o conceito de Document Object Model



Agenda

- DOM
- Seleção e manipulação de elementos
- Eventos



DOM. O que é?

- O DOM (Document Object Model) é uma interface de programação que representa a estrutura de um documento HTML/XML em forma de uma árvore de objetos.
- O DOM permite que você acesse e manipule os elementos, atributos e conteúdos do documento por meio de scripts em JavaScript.

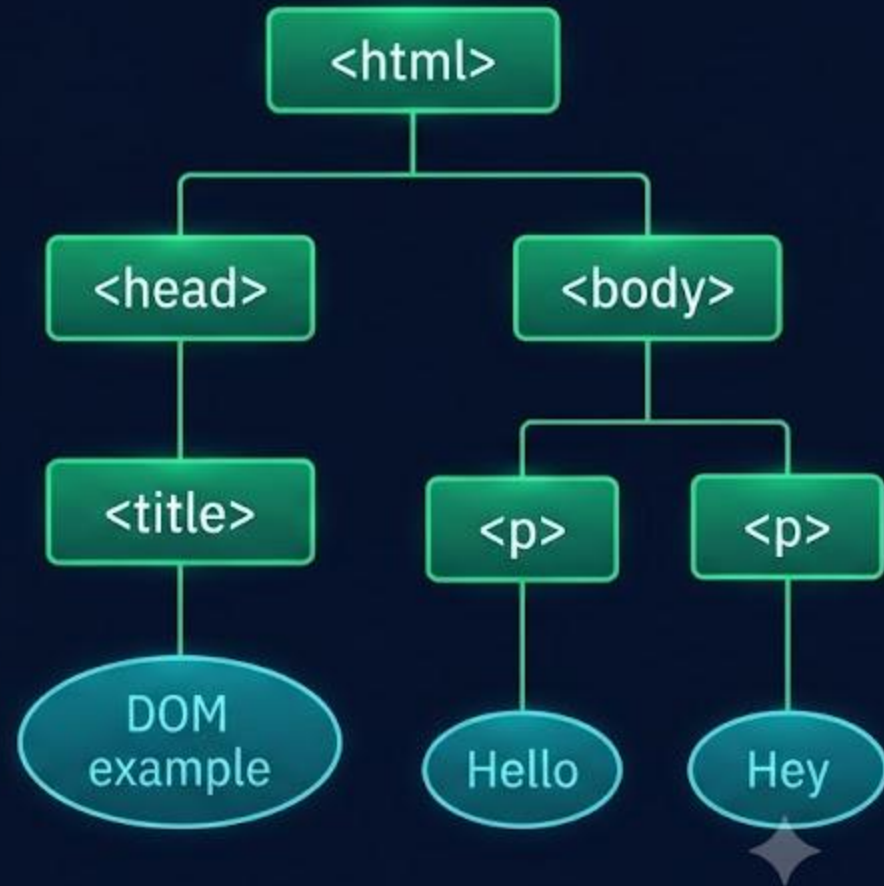


Como funciona:

Basicamente, o DOM transforma o documento HTML/XML em uma estrutura de objetos, onde cada elemento, atributo e texto é representado por um nó na árvore de objetos.

Essa representação em forma de árvore permite que você navegue pelos elementos, acesse suas propriedades, altere seu conteúdo e estilize-os dinamicamente.

```
1 <html>
2 <head>
3   <title>
4     DOM example
5   </title>
6 </head>
7 <body>
8   <p id = "para1">Hello</p>
9   <p id = "para2">Hey</p>
10 </body>
11 </html>
```



DOM TREE STRUCTURE VISUALIZATION



Seleção e manipulação de elementos

Para selecionar elementos no DOM, você pode usar métodos e propriedades disponíveis no JavaScript.

Alguns métodos

Método	Descrição
getElementById(id)	Retorna o elemento com o ID especificado.
getElementsByClassName(className)	Retorna uma coleção de elementos com a classe especificada.
getElementsByTagName(tagName)	Retorna uma coleção de elementos com a tag especificada.
querySelector(selector)	Retorna o primeiro elemento que corresponde ao seletor CSS especificado.
querySelectorAll(selector)	Retorna uma lista de todos os elementos que correspondem ao seletor CSS especificado.

Manipulando eventos

- Uma vez selecionado o elemento desejado, você pode manipulá-lo de várias maneiras, como:
 - Acessar e modificar propriedades, como `innerHTML`, `textContent`, `value`, `src`, `href`, entre outras.
 - Adicionar ou remover classes com as propriedades `classList` e `className`.
 - Modificar estilos utilizando a propriedade `style`.
 - Adicionar ou remover atributos utilizando as propriedades `setAttribute` e `removeAttribute`.
 - Manipular eventos, como clique, submissão de formulário, `mouseover`, etc.



Eventos

- Os eventos em JavaScript são mecanismos que permitem ao seu programa responder a interações do usuário ou a mudanças no documento.
 - Quando uma ação ocorre, como um clique em um botão, um evento é disparado, permitindo a execução de uma função específica.
- Para capturar e responder a essas ações, utiliza-se um "ouvinte de eventos" (event listener), que é registrado em um elemento específico. Existem diversas formas de implementar esse registro no código.

Exemplo 1

- Utilizando a propriedade on<evento> diretamente no elemento HTML.
- Usando o método `addEventListener` para adicionar ouvintes de eventos de forma mais flexível.

javascript

```
let button = document.querySelector('button');

button.addEventListener('click', function() {
  console.log('O botão foi clicado!');
});
```

Nesse exemplo, um ouvinte de evento de clique é adicionado ao botão selecionado. Quando o botão for clicado, a função passada como argumento será executada, exibindo uma mensagem no console.



Nota

- Apresenta-se, nos próximos slides, 10 exemplos de uso dos métodos de seleção do DOM, acompanhados de trechos de código HTML para ilustrar cada caso.
- Estão descritos o que se pretende fazer e, em seguida, é apresentado o documento HTML correspondente.

Eventos e Manipulação do DOM



Exemplo 1

- Exemplo usando `getElementById(id)`: Pretendemos selecionar um elemento de uma página pelo seu ID.

```
html Copy code  
  
<!DOCTYPE html>  
<html>  
  <head>  
    <title>Exemplo getElementById</title>  
  </head>  
  <body>  
    <h1 id="titulo">Titulo</h1>  
    <script>  
      let elemento = document.getElementById("titulo");  
      console.log(elemento.innerText); // Exibe o texto "Título" no consola  
    </script>  
  </body>  
</html>
```

Exemplo 2

- Exemplo usando `getElementsByClassName(className)`: Pretendemos selecionar todos os elementos de uma página que possuam uma determinada classe.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo getElementByClassName</title>
  </head>
  <body>
    <p class="paragrafo">Parágrafo 1</p>
    <p class="paragrafo">Parágrafo 2</p>
    <script>
      let elementos = document.getElementsByClassName("paragrafo");
      for (let i = 0; i < elementos.length; i++) {
        console.log(elementos[i].innerText);
      }
      // Exibe "Parágrafo 1" e "Parágrafo 2" no console
    </script>
  </body>
</html>
```

Exemplo 3

- Exemplo usando `getElementsByClassName(className)`: Pretendemos selecionar todos os elementos de uma página que possuam uma determinada classe

```
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo getElementByClassName</title>
  </head>
  <body>
    <p class="paragrafo">Parágrafo 1</p>
    <p class="paragrafo">Parágrafo 2</p>
    <script>
      let elementos = document.getElementsByClassName("paragrafo");
      for (let i = 0; i < elementos.length; i++) {
        console.log(elementos[i].innerText);
      }
      // Exibe "Parágrafo 1" e "Parágrafo 2" no console
    </script>
  </body>
</html>
```

Exemplo 3

- Exemplo usando `getElementsByName(tagName)`: Pretendemos selecionar todos os elementos de uma página que tenham uma determinada tag.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo getElementByTagName</title>
  </head>
  <body>
    <h2>Subtítulo</h2>
    <p>Parágrafo 1</p>
    <p>Parágrafo 2</p>
    <script>
      let elementos = document.getElementsByTagName("p");
      for (let i = 0; i < elementos.length; i++) {
        console.log(elementos[i].innerText);
      }
      // Exibe "Parágrafo 1" e "Parágrafo 2" no console
    </script>
  </body>
</html>
```

Exemplo 4

- Exemplo usando `querySelector(selector)`: Pretendemos selecionar o primeiro elemento de uma página que corresponda a um seletor CSS específico.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo querySelector</title>
  </head>
  <body>
    <div class="container">
      <h3>Título</h3>
      <p>Conteúdo do parágrafo</p>
    </div>
    <script>
      let elemento = document.querySelector(".container p");
      console.log(elemento.innerText); // Exibe "Conteúdo do parágrafo"
    </script>
  </body>
</html>
```

Exemplo 5

Exemplo usando `querySelectorAll(selector)`:
Pretendemos selecionar todos os elementos de uma página que correspondam a um seletor CSS específico.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo querySelectorAll</title>
  </head>
  <body>
    <ul>
      <li>Item 1</li>
      <li>Item 2</li>
      <li>Item 3</li>
    </ul>
    <script>
      let elementos = document.querySelectorAll("li");
      for (let i = 0; i < elementos.length; i++) {
        console.log(elementos[i].innerText);
      }
      // Exibe "Item 1", "Item 2" e "Item 3" no console
    </script>
  </body>
</html>
```

Exemplo 6

Exemplo usando `getElementById(id)` com manipulação de conteúdo: Pretendemos selecionar um elemento pelo seu ID e alterar seu conteúdo.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo getElementById com manipulação de conteúdo</title>
  </head>
  <body>
    <h1 id="titulo">Titulo</h1>
    <script>
      let elemento = document.getElementById("titulo");
      elemento.innerText = "Novo Título";
    </script>
  </body>
</html>
```

Exemplo 7

Exemplo usando `getElementsByClassName(className)` com manipulação de estilo: Pretendemos selecionar todos os elementos de uma classe e alterar seu estilo.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo getElementsByClassName com manipulação de estilo</title>
    <style>
      .paragrafo {
        color: blue;
      }
    </style>
  </head>
  <body>
    <p class="paragrafo">Parágrafo 1</p>
    <p class="paragrafo">Parágrafo 2</p>
    <script>
      let elementos = document.getElementsByClassName("paragrafo");
      for (let i = 0; i < elementos.length; i++) {
        elementos[i].style.fontWeight = "bold";
      }
    </script>
  </body>
</html>
```

Exemplo 8

Exemplo usando `getElementsByTagName(tagName)` com manipulação de atributos: Pretendemos selecionar todos os elementos de uma tag e alterar um atributo específico.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo getElementByTagName com manipulação de atributos</title>
  </head>
  <body>
    
    
    <script>
      let elementos = document.getElementsByTagName("img");
      for (let i = 0; i < elementos.length; i++) {
        elementos[i].setAttribute("alt", "Imagem");
      }
    </script>
  </body>
</html>
```

Exemplo 9

Exemplo usando `querySelector(selector)` com manipulação de classes: Pretendemos selecionar o primeiro elemento correspondente a um seletor CSS e adicionar/remover classes.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo querySelector com manipulação de classes</title>
    <style>
      .destaque {
        color: red;
      }
    </style>
  </head>
  <body>
    <p>Parágrafo</p>
    <script>
      let elemento = document.querySelector("p");
      elemento.classList.add("destaque"); // Adiciona a classe "destaque"
      // elemento.classList.remove("destaque"); // Remove a classe "destaque"
    </script>
  </body>
</html>
```

Exemplo 10

Exemplo de uso de eventos com `addEventListener`:
Pretendemos adicionar um ouvinte de evento a um elemento para responder a um evento específico.

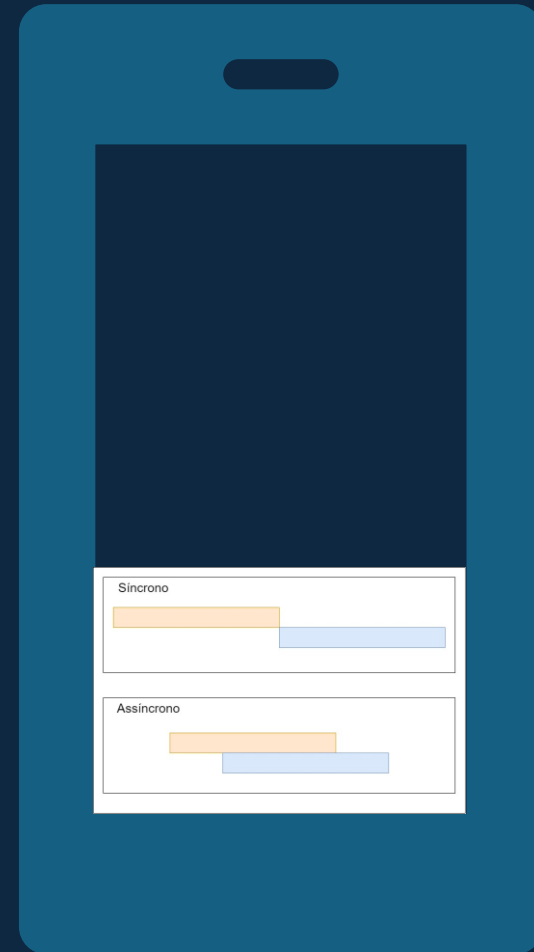
```
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo de uso de eventos</title>
  </head>
  <body>
    <button id="botao">Clique aqui</button>
    <script>
      let botao = document.getElementById("botao");
      botao.addEventListener("click", function() {
        alert("O botão foi clicado!");
      });
    </script>
  </body>
</html>
```

Conclusão

- DOM é essencial para a criação de páginas web interativas, dinâmicas e personalizadas.
- Ele permite que os desenvolvedores acessem, manipulem e interajam com os elementos de uma página de forma programática, abrindo um leque de possibilidades para a criação de experiências ricas e funcionais.
- O conhecimento do DOM é fundamental para qualquer desenvolvedor web, pois é a base para o desenvolvimento de aplicações web modernas e responsivas.

Javascript – Assíncrono

Material
MOBILE_002



Javascript Assíncrono - Agenda

- Callbacks
- Promessas
- Async/Await
- Fetch API e
- Ajax

Callbacks

- **Callbacks** são funções que são passadas como argumentos para outras funções e são executadas após a conclusão de uma operação assíncrona.
- Elas são amplamente utilizadas em JavaScript para lidar com eventos e operações que levam tempo

Callbacks - Exemplo

```
function obterDados(callback) {  
  // Simulando uma operação assíncrona  
  setTimeout(function() {  
    const dados = 'Dados obtidos';  
    callback(dados);  
  }, 2000);  
}  
  
function mostrarDados(dados) {  
  console.log(dados);  
}  
  
obterDados(mostrarDados); // Chama a função obterDados com a função mostrarDados
```

Sobre o exemplo de Callback

- Nesse exemplo, a função **obterDados** simula uma operação assíncrona com **setTimeout** e, após 2 segundos, chama o callback **mostrarDados** passando os dados obtidos como argumento.

Promessas

- Promessas são uma maneira mais moderna de lidar com operações assíncronas em JavaScript.
- Elas representam um valor que **pode estar disponível agora, no futuro ou nunca**.
- As promessas podem estar em três estados:
 - **pendente,**
 - **resolvida (com sucesso) ou**
 - **rejeitada (com erro).**

Exemplo de promessas

```
function obterDados() {  
  return new Promise(function(resolve, reject) {  
    // Simulando uma operação assíncrona  
    setTimeout(function() {  
      const dados = 'Dados obtidos';  
      if (dados) {  
        resolve(dados);  
      } else {  
        reject('Erro ao obter dados');  
      }  
    }, 2000);  
  });  
}
```

```
obterDados()  
  .then(function(dados) {  
    console.log(dados);  
  })  
  .catch(function(erro) {  
    console.log(erro);  
  });
```

Sobre o exemplo anterior

- Nesse exemplo, a função **obterDados** retorna uma promessa que simula uma operação assíncrona.
- Se a operação for **concluída com sucesso, a promessa é resolvida** chamando `resolve` com os dados.
- Caso **contrário, a promessa é rejeitada chamando `reject`** com uma mensagem de erro.
- O método **then** é usado para tratar a resolução da **promessa** e o **método catch** para tratar a rejeição.

Async/Await:

- Async/Await é uma sintaxe mais moderna introduzida no ES2017 para simplificar a escrita de código assíncrono.
- Permite escrever código assíncrono como se fosse síncrono, o que torna a leitura e a escrita mais intuitivas.
- Async é usado para declarar uma função assíncrona e Await é usado para esperar que uma promessa seja resolvida antes de continuar a execução.

Exemplo Async/await

- Nesse exemplo, a função **mostrarDados** é declarada como assíncrona usando a palavra-chave **async**.
- Dentro dessa função, usamos **await** antes da chamada da função **obterDados** para esperar que a promessa seja resolvida antes de prosseguir. Os erros são tratados com um bloco try/catch.

```
function obterDados() {  
  return new Promise(function(resolve, reject) {  
    // Simulando uma operação assíncrona  
    setTimeout(function() {  
      const dados = 'Dados obtidos';  
      if (dados) {  
        resolve(dados);  
      } else {  
        reject('Erro ao obter dados');  
      }  
    }, 2000);  
  });  
}
```

```
async function mostrarDados() {  
  try {  
    const dados = await obterDados();  
    console.log(dados);  
  } catch (erro) {  
    console.log(erro);  
  }  
}
```

```
mostrarDados();
```

Fetch API e Ajax

- A Fetch API é uma API nativa do JavaScript que fornece uma interface para fazer requisições HTTP assíncronas.
- Ela é mais moderna e flexível em comparação com a abordagem tradicional com Ajax, que envolve o uso da API XMLHttpRequest.

Exemplo

```
fetch('https://api.exemplo.com/dados')  
  .then(function(response) {  
    return response.json();  
  })  
  .then(function(data) {  
    console.log(data);  
  })  
  .catch(function(error) {  
    console.log(error);  
  });
```



Sobre o exemplo Fetch API e Ajax:

- Nesse exemplo, usamos o método **fetch** para fazer uma requisição HTTP GET para a URL fornecida.
- Em seguida, encadeamos os métodos **then** para lidar com a resposta da requisição.
- No primeiro **then**, chamamos o método `json` para obter os dados no formato JSON.
- No segundo **then**, recebemos os dados e podemos fazer o que for necessário. O método **catch** é usado para lidar com erros.

Laboratório

- ▶ Vamos criar um exemplo simulando operações assíncronas utilizando a Promessa (Promise) e explicar passo a passo o que está acontecendo.
- ▶ Vamos simular uma chamada assíncrona a uma API para **obter dados de um usuário do Github**.

Crie o arquivo script.js

```
1  function obterDadosUsuario(nomeUsuario) {  
2      const url = `https://api.github.com/users/${nomeUsuario}`;  
3  
4      return fetch(url)  
5          .then(function (response) {  
6              if (response.ok) {  
7                  return response.json();  
8              } else {  
9                  throw new Error('Erro ao obter dados do usuário');  
10             }  
11         })  
12         .then(function (data) {  
13             return data;  
14         })  
15         .catch(function (error) {  
16             console.log('Erro:', error);  
17         });  
18  }  
19
```

```
20 obterDadosUsuario('maromo71')
21   .then(function (usuario) {
22     |   exibirDadosUsuario(usuario);
23   });
24
25 function exibirDadosUsuario(usuario) {
26   const tableBody = document.querySelector('#dados-table tbody');
27
28   const row = document.createElement('tr');
29   const nomeCell = document.createElement('td');
30   const reposCell = document.createElement('td');
31   const seguidoresCell = document.createElement('td');
32
33   nomeCell.textContent = usuario.name;
34   reposCell.textContent = usuario.public_repos;
35   seguidoresCell.textContent = usuario.followers;
36
37   row.appendChild(nomeCell);
38   row.appendChild(reposCell);
39   row.appendChild(seguidoresCell);
40
41   tableBody.appendChild(row);
42 }
```

Sobre o arquivo script.js

- Este exemplo ilustra como consultar a API do GitHub para obter dados de um usuário (ex: "maromo71"):
 - A função **obterDadosUsuario** utiliza o método fetch para realizar uma requisição GET à API.
 - O tratamento da resposta ocorre via encadeamento de métodos: **then** processa o sucesso (convertendo para JSON) e **catch** lida com eventuais erros.
 - Após a chamada da função, os dados retornados são processados e exibidos no console.

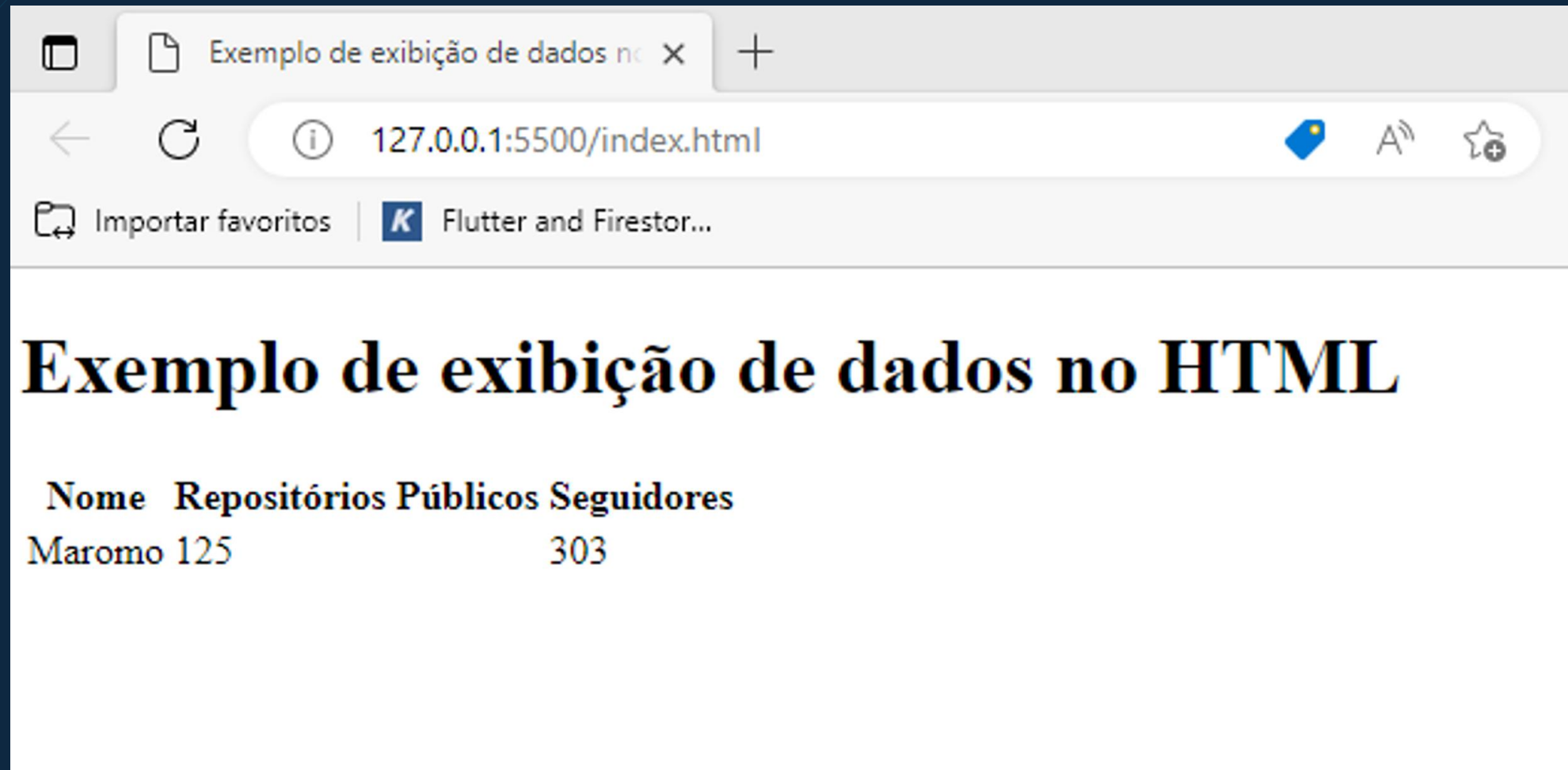
Arquivo index.html

```
index.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>Exemplo de exibição de dados no HTML</title>
5  </head>
6  <body>
7    <h1>Exemplo de exibição de dados no HTML</h1>
8
9    <table id="dados-table">
10     <thead>
11     <tr>
12       <th>Nome</th>
13       <th>Repositórios Públicos</th>
14       <th>Seguidores</th>
15     </tr>
16   </thead>
17   <tbody>
18     <!-- Os dados serão adicionados aqui dinamicamente -->
19   </tbody>
20 </table>
21
22   <script src="script.js"></script>
23 </body>
24 </html>
25
```

Arquivo index.html - explicação

- **HTML:** Criamos uma tabela (id="dados-table") com um cabeçalho fixo e um corpo vazio, reservado para receber os dados dinamicamente.
- **JavaScript:** A função obterDadosUsuario consulta a API do GitHub utilizando o nome do usuário como parâmetro.
- **Requisição:** O método fetch realiza a busca dos dados. Se a requisição for bem-sucedida (status 200), o retorno é convertido para o formato JSON.
- **Exibição:** O objeto com os dados obtidos é enviado para a função exibirDadosUsuario, que cuida de preencher a tabela na página

Resultado da Execução



Conclusão

- O domínio da programação assíncrona é indispensável para o desenvolvimento web atual.
- O uso estratégico de `promises`, `async/await` e `Fetch API` simplifica a execução de tarefas demoradas, tornando o código não apenas mais legível, mas significativamente mais robusto e performático para o usuário final.



Agora você será capaz de:

- Interagir com o navegador: Selecionar e alterar qualquer parte de uma página HTML usando JavaScript.
- Criar páginas dinâmicas: Responder a cliques, digitação e movimentos do mouse.
- Gerenciar tarefas demoradas: Executar processos em segundo plano sem travar a interface.
- Conectar-se com o mundo: Buscar dados de APIs externas (como a do GitHub) e exibí-los dinamicamente no seu projeto.

Fim



Prof. Maromo